AgentBPF: A Reference Architecture for Runtime Policy Enforcement in Agentic AI Systems

Marut Pandya
pandyamarut@gmail.com

March 17, 2026

Abstract

Agentic AI systems—autonomous software agents that iteratively observe their environment, plan actions, and execute tools—are rapidly moving from research prototypes to production deployments. Yet the security and governance infrastructure for these systems remains fragmented: existing work addresses point problems such as prompt injection detection, sandboxing, or access control, but does not define a unified runtime enforcement architecture for autonomous tool use. We argue that the critical missing boundary in agentic systems lies between language-model reasoning and tool execution.

We present **AgentBPF**, a conceptual framework and reference architecture for runtime policy enforcement in agentic systems, drawing an explicit architectural parallel with eBPF's role in Linux kernel security. AgentBPF defines a programmable control plane at the tool-call boundary, composed of four operational primitives: the Tool Invocation Intercept (TII), Context Taint Tracker (CTT), Semantic Intent Classifier (SIC), and Agent Audit Ring Buffer (AARB). We further introduce the *Capability Manifest*, a declarative least-privilege policy interface, and the *Autonomous Action Budget*, an operational bound on unchecked autonomous execution before human checkpointing.

The contribution of this paper is *architectural* rather than algorithmic. We define the enforcement model, policy interfaces, and runtime abstractions required to make tool-using agents governable in production; position them relative to concurrent work in agent safety, taint analysis, and systems security; and identify the principal implementation and research questions needed to realize this design in practice.

1 Introduction

The emergence of agentic AI systems marks a qualitative shift in how software interacts with the world. Where previous generations of AI produced text for human consumption, modern agent frameworks increasingly execute real actions: reading and writing files, sending messages, querying databases, controlling browsers, and calling arbitrary APIs. These systems operate across multi-step workflows, maintain state, and adapt plans based on intermediate results.

This transition from responsive to autonomous software creates a governance crisis. The security models underlying operating systems, cloud platforms, and enterprise IT infrastructure generally assume a *human principal* who directs each significant action and can be held accountable for it. Agentic AI systems violate that assumption. An agent may take hundreds of actions on behalf of a

user who specified only a high-level goal, while the causal chain from user intent to system action is mediated by a stochastic language model.

The threat landscape is concrete rather than hypothetical. Prompt injection attacks—where an adversary embeds instructions in data the agent processes, such as emails, webpages, or documents—have been used to induce credential exfiltration, unauthorized downstream actions, and cryptocurrency wallet drains [8, 5]. Existing defenses remain fragmented: some focus on detecting malicious content [10, 16], others on constraining tool use [12], and others on training-time alignment [1]. None defines a unified runtime control plane at the boundary where reasoning becomes action.

The eBPF analogy. We argue that the current state of agentic AI security is analogous, in architectural terms, to Linux kernel security before eBPF. In that era, adding observability or security policy to kernel operations required invasive modification, kernel modules with attendant stability risks, or prohibitive operational complexity. eBPF changed that by introducing programmable hook points, a safe extension model, and first-class telemetry—with the result being not merely better security, but a general control-plane abstraction that unlocked an ecosystem of tools (Cilium, Falco, Pixie, Tetragon) that would have been impossible otherwise [9].

We therefore present AgentBPF as a reference architecture for runtime policy enforcement in agentic systems. The analogy is *architectural rather than guarantee-equivalent*: unlike the eBPF kernel verifier, which provides mathematical safety proofs, AgentBPF's semantic components are probabilistic enforcement guards. The contribution is the design philosophy applied one abstraction level above the OS kernel: a programmable, low-overhead enforcement layer at the agent tool-call boundary, with a policy language, telemetry backbone, and composable primitives.

Contributions. This paper makes the following contributions:

- An **architectural formulation** of the agent security problem, identifying the LLM-to-tool boundary as the critical missing enforcement point.
- A **reference enforcement model** composed of four runtime primitives (TII, CTT, SIC, AARB) whose novelty is their architectural composition rather than any single primitive in isolation.
- A **policy abstraction**—the Capability Manifest—and the Autonomous Action Budget as a hard operational invariant bounding unchecked agent autonomy.
- A **threat-to-enforcement mapping** connecting prompt injection, credential exfiltration, and goal drift to specific AgentBPF enforcement points.
- A **situated positioning** relative to concurrent work (RTBAS, AgentArmor, Attention Tracker, Microsoft governance toolkit), and a **research agenda** for open problems in persistent taint, policy verification, and multi-agent trust.

2 Background and Motivation

2.1 Architecture of Current Agentic Systems

A canonical agent architecture consists of five components arranged in an observe-plan-act loop. Table 1 summarises their security relevance. The critical gap lies between the LLM Core and the Tool Dispatcher: current systems have no enforcement layer at this boundary beyond basic schema

validation. There is no analog to the Linux kernel's capability system, no enforcement of least privilege, and no structured audit log of agent actions.

Component	Security Relevance
Perception	Primary injection surface—all untrusted data enters here
Context Store	Taint propagation—untrusted data contaminates reasoning
LLM Core	Non-deterministic action generation from full context
Tool Dispatcher	**Critical enforcement point**—last programmatic gate
Tool Executors	Execution plane—where real-world effects occur

Table 1: Agent architecture components and security relevance. The gap between LLM Core and Tool Dispatcher is where AgentBPF operates.

2.2 The eBPF Precedent

eBPF allows sandboxed programs to be attached to kernel hook points without modifying kernel source code. Programs are compiled to eBPF bytecode, verified (proving termination and memory safety), and JIT-compiled to native code. The key design insight is *separation of mechanism from policy*: the kernel provides hook points; operators supply policy programs. This enables policy without privilege, composable layered enforcement, and observability as a first-class primitive. Table 2 maps eBPF concepts to their AgentBPF equivalents.

Linux / eBPF	**AgentBPF Equivalent**
Kernel	Tool execution infrastructure
Syscall	Tool invocation
kprobe / LSM hook	Tool Invocation Intercept (TII)
seccomp-bpf	Structural Policy / Capability Manifest
LSM program	Semantic Intent Classifier (SIC)
perf ring buffer	Agent Audit Ring Buffer (AARB)
BPF maps	Policy context store
eBPF verifier	Manifest validator + SIC confidence gate
Falco / Tetragon	Future AgentBPF tooling ecosystem

Table 2: eBPF-to-AgentBPF concept mapping. The analogy is architectural, not guarantee-equivalent: the eBPF verifier provides mathematical proofs; the SIC provides probabilistic bounds.

2.3 Why Existing Approaches Are Insufficient

A growing body of work addresses agentic AI security, but no existing system provides the unified enforcement architecture AgentBPF proposes.

RTBAS [16] is the closest academic precursor. It applies information flow control and taint-inspired techniques to Tool-Based AI Systems, benchmarked on AgentDojo [6]. Its core insight—that classical

dynamic taint analysis (DTA) cannot apply to LLM contexts due to their opaque, dynamic nature—directly motivates the CTT design. However, RTBAS addresses injection detection in isolation. It proposes no policy enforcement architecture, no capability manifest language, no action budget primitive, and no audit telemetry backbone. AgentBPF's CTT builds on RTBAS's foundations while situating taint propagation within a broader enforcement pipeline.

Attention Tracker [10] demonstrates that prompt injection produces a measurable *distraction effect* in LLM attention patterns—specific heads shift focus from original to injected instructions—enabling training-free detection with AUROC improvements up to 10% over prior methods. This work provides the empirical foundation for attention-weight-based taint signal in white-box CTT deployments. Critically, it is a *detection* method, not a policy enforcement mechanism: it identifies injection but provides no infrastructure to block, escalate, or audit based on that signal within an execution loop.

AgentArmor [18] abstracts agent execution traces into Program Dependence Graphs (PDGs) to enable formal taint analysis, correctly identifying that cross-resource taint propagation (write-read chains, cache pollution, persistent memory) is deeper than single-round detection. For white-box deployments, AgentArmor's PDG formalization is more rigorous than CTT. AgentBPF targets a different deployment reality: most production LLM deployments are black-box API calls where runtime trace interception is unavailable.

Microsoft's governance toolkit [12] is the most complete existing implementation of structural policy enforcement: sub-millisecond evaluation, zero-trust identity (Ed25519/SPIFFE), and OWASP Agentic Top 10 coverage across 12+ frameworks. AgentBPF's structural policy layer is conceptually aligned with this toolkit. The differentiation is the taint-conditioned semantic layer (SIC), the Autonomous Action Budget as a hard security invariant, and the integrated AARB audit backbone.

Training-time alignment [1] reduces the probability of harmful behavior but provides no hard runtime guarantees. A well-aligned model remains susceptible to adversarial injection.

The pattern across existing approaches is clear: each addresses one layer— detection, or structural enforcement, or formal trace analysis—without providing the integrated pipeline connecting taint provenance through structural policy to semantic verification to audit telemetry. **AgentBPF's contribution is the architecture of integration.**

3 The AgentBPF Framework

Before presenting the framework, we clarify its scope. AgentBPF is not an empirical solution claiming universal security guarantees. It is a *systems design proposal*: a specification of the enforcement boundary, policy interfaces, and runtime abstractions required for governable agent execution, in a form that can guide implementations and future empirical evaluation. Figure 1 shows the enforcement pipeline.

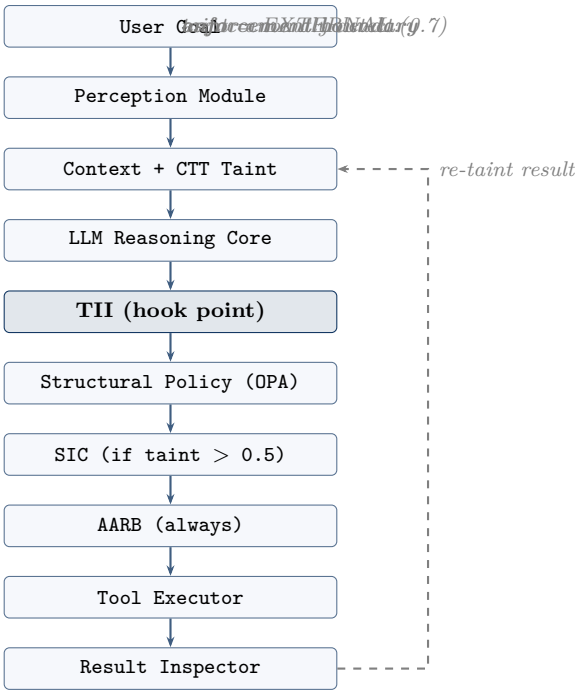

Figure 1: AgentBPF enforcement pipeline. Every tool call passes synchronously through the TII before execution. The taint score threads from CTT through structural policy, SIC invocation, and AARB anomaly thresholds.

3.1 Tool Invocation Intercept (tii)

The TII is the primary hook point of AgentBPF, analogous to eBPF attachment points (kprobes, uprobes, LSM hooks). It intercepts every tool call generated by the LLM reasoning core before it reaches the executor.

Definition 1 (TII Event)**.** A tool call at the TII is a 5-tuple $\langle\, name,\ args,\ \tau,\ agent,\ t\,\rangle$, where $name \in \Sigma_{tools}$, $args$ is a typed argument map, $\tau \in [0, 1]$ is the CTT taint score, $agent$ is a UUID principal, and t is a Unix nanosecond timestamp.

The TII routes each event through the policy pipeline and returns a verdict $v \in \{\text{ALLOW}, \text{BLOCK}, \text{ESCALATE}\}$. ALLOW executes the tool; BLOCK rejects it with an error returned to the agent LLM; ESCALATE pauses the agent loop, notifies the human operator, and resumes on an explicit decision.

The TII is *synchronous*—it blocks the agent loop until a verdict is returned. Advisory-only checks can be bypassed by a sufficiently capable model; synchrony is necessary for hard enforcement. The TII also instruments tool results as a return probe, recording the world-state delta (files created, messages sent, APIs called).

Integration patterns. *Sidecar proxy* (recommended for OpenClaw, AutoGen): AgentBPF runs as a local gRPC service on `127.0.0.1:18790`. The agent's tool dispatcher proxies all calls through AgentBPF before execution. Zero framework modification required; ∼2–5 ms local overhead. *SDK middleware*: A Python/Node.js library wraps the tool dispatcher inline. Lower overhead (∼0.5 ms), requires framework integration. *MCP gateway*: For MCP-native deployments, AgentBPF implements the MCP gateway protocol, sitting between the agent and all MCP servers.

3.2 Context Taint Tracker (ctt)

The CTT maintains a taint metadata annotation alongside the agent's context window, propagating trust-level information from data sources through agent reasoning to tool call verdicts.

Theoretical positioning. In classical security, DTA tags data from untrusted sources and propagates the tag through data transformations, blocking execution at sensitive sinks [4, 14]. For LLM-based agents, this approach does not transfer directly. RTBAS [16] identifies the core difficulty: LLM agents operate in dynamic and opaque environments where the relationship between input data and resulting tool calls is mediated by continuous attention over the entire context—not a discrete, traceable data flow. AgentArmor [18] provides a more rigorous PDG-based formalization for white-box settings.

Hung et al. [10] (Attention Tracker) provide the key empirical foundation: they demonstrate that prompt injection produces a consistent *distraction effect* in attention patterns—specific heads shift focus from original to injected instructions in a measurable, model-agnostic way. The CTT's contribution is not taint detection (which prior work addresses) but *enforcement integration*: transforming a detection signal into a runtime score that conditions the full AgentBPF pipeline.

Taint taxonomy. We define a four-level taint taxonomy $\mathcal{T}$ as shown in Table 3.

Level	τ	Source & Policy Trigger
TRUSTED	0.0	System prompt / direct user messages; full autonomy
DERIVED	0.3	Agent persistent memory; standard structural policy
EXTERNAL	0.7	Tool results, API responses, web content; SIC engaged
HOSTILE	1.0	Injection-flagged; all calls ESCALATE

Table 3: Taint taxonomy. Higher taint score triggers increasing policy scrutiny.

Composite taint score. When the LLM generates a tool call, the CTT computes:

$$\tau_{call} = \frac{\sum_i w_i \cdot \tau_i}{\sum_i w_i}, \quad w_i = e^{-\lambda \cdot d_i} \tag{1}$$

where d_i is the token distance of segment i from the most recent token and $\lambda = 0.01$ is a configurable decay constant. This recency weighting is a tractable approximation for black-box deployments. For white-box deployments with attention access, w_i should be replaced with the actual attention weight from the last token to segment i, following Hung et al.'s [10] distraction-effect methodology.

Result-side injection detection. Tool results re-enter the agent context and present a secondary injection surface. The CTT runs a fast binary classifier on all tool results before they are added to context, tagging injection candidates as HOSTILE ($\tau = 1.0$). This classifier is a fine-tuned 125M-parameter encoder model trained on labeled injection examples from security research disclosures.

3.3 Semantic Intent Classifier (sic)

The SIC is AgentBPF's equivalent of Linux Security Module hooks: policy enforcement based on *semantic understanding* of the requested action, not merely its structural properties. Structural policy can enforce "this agent may not call `exec_shell`"; it cannot enforce "this email send is consistent with the user's stated goal." The SIC addresses the latter.

RTBAS [16] uses an LM-judge screener for a related purpose. The SIC differs in three ways: (1) it is conditioned on the full session `user_goal`—not just local task context—enabling detection of *goal-drift attacks* where an agent is gradually redirected from its original purpose across many steps; (2) it is invoked selectively (see below), keeping it off the critical path for the majority of low-risk calls; (3) it receives τ_{call} as an explicit input, allowing the classifier to calibrate confidence thresholds based on context provenance.

The SIC is implemented as a lightweight specialized language model (1–3B parameters, fine-tuned on labeled agent action trajectories). Input schema:

```
sic_input := {
  user_goal:      string,        -- full session goal
  action_history: List[call],   -- last N tool calls
  proposed:       tool_call,
  taint_score:    float,
}
sic_verdict := {
  decision:   ALLOW | BLOCK | ESCALATE,
  confidence: float,             -- in [0, 1]
  reason:     string,
  policy_ref: string | null,
}
```

Listing 1: SIC input and verdict schema.

Selective invocation. The SIC fires only when: (1) $\tau_{call} \geq 0.5$ (external content influenced the decision); (2) the tool is high-risk (`send_email`, `exec_shell`, `file_delete`, `http_post`); (3) the agent has reached its Autonomous Action Budget checkpoint; or (4) the AARB anomaly detector has flagged the trajectory. This selective invocation ensures the SIC's ∼30–50 ms latency does not impact the majority of low-risk, low-taint operations.

Training data strategy. *Phase 1 (Bootstrap):* Synthetic adversarial trajectories labeled by security researchers. Target: 50K examples per class. *Phase 2 (Flywheel):* AARB audit logs with human approval/rejection decisions as ground truth labels—the enforcement infrastructure and classifier training reinforce each other. *Phase 3 (Domain adaptation):* Per-deployment fine-tuning on organization-specific action vocabularies.

3.4 Structural Policy Engine and Capability Manifest

The Structural Policy Engine is the fast, deterministic enforcement layer that runs before the SIC—analogous to seccomp-bpf in Linux. It evaluates tool calls against a *Capability Manifest*: a declarative, default-deny specification of permitted agent actions.

Invariant 1 (Default Deny)**.** Any tool not explicitly listed in the Capability Manifest is BLOCKed. The manifest is the permission grant, not the exception list.

```
version: 1
agent_id:   email-assistant-v1
principal:  user@company.com

capabilities:
  - tool: read_email
    constraints:
      - max_per_hour: 100
      - folders: [inbox, sent]
  - tool: send_email
    constraints:
      - allowed_recipients: ["*@company.com"]
      - require_confirmation_if:
```

```
          taint_score_gt: 0.5
  - tool: exec_shell
    allow: false

budget:
  autonomous_actions_before_checkin: 50
  cost_usd_per_day: 10.00

taint_policy:
  escalate_threshold: 0.7
  block_threshold:    1.0
```

Listing 2: Example Capability Manifest (YAML).

Autonomous Action Budget. The `budget` block introduces a primitive we term the *Autonomous Action Budget*: a finite token of unchecked autonomous actions per session, after which the agent must checkpoint with the human operator.

Invariant 2 (Action Budget)**.** Let n_t be the count of ALLOWed actions in session t. When $n_t \bmod B = 0$ for budget B, the next tool call receives verdict ESCALATE regardless of other policy, until explicit human confirmation.

This provides a hard upper bound on the blast radius of any successful injection attack: regardless of how deeply an adversary has compromised the agent's context, the agent is forced to surface for human review every B actions. The correct calibration of B is an open problem (Section 6).

3.5 Agent Audit Ring Buffer (aarb)

The AARB is AgentBPF's telemetry backbone, analogous to eBPF's perf ring buffer. It records every tool invocation as a structured event with low overhead, regardless of verdict. Full argument logging is opt-in (privacy); by default, the AARB records argument hashes and LLM-generated one-sentence summaries.

Behavioral anomaly detection. AARB events stream to an async anomaly detection pipeline: (1) a trajectory encoder produces a 256-dim embedding of the agent's last 50 actions; (2) a per-agent baseline is maintained as a rolling Gaussian in embedding space; (3) anomaly score = Mahalanobis distance from the baseline centroid; (4) score above threshold triggers ESCALATE on the next tool call. This catches behavioral patterns that individually pass structural policy—a sudden shift from `read_email` to credential path reads and `http_post` to an external domain is anomalous even if each call is within manifest constraints.

4 Threat Model Coverage

Threat	AgentBPF Mitigation
Indirect prompt injection	CTT marks ingested content EXTERNAL; SIC verifies intent alignment; AARB logs injection pattern
Credential exfiltration	Manifest scopes `file_read` to workspace; `http_post` blocklisted or confirmation-gated
Autonomous scope creep	Action Budget forces checkin every B actions; SIC verifies alignment with session goal on high-risk calls
Malicious skill package	TII intercepts all calls regardless of skill origin; AARB anomaly detector flags unusual patterns per skill ID
Agent swarm exploitation	Inter-agent messages tagged EXTERNAL (τ=0.7); manifests are per-agent; no trust transitivity
Goal drift (long-horizon)	SIC conditioned on original `user_goal`; detects gradual redirection across many steps

Table 4: Threat-to-enforcement mapping for AgentBPF.

5 Performance Targets

Component	p99	Notes
TII routing	<0.5 ms	Stateless local socket
Structural policy	<2 ms	In-memory OPA evaluation
CTT score	<1 ms	Recency-weighted dot product
SIC (invoked)	<50 ms	1–3B model, GPU sidecar
AARB write	<0.1 ms	Lock-free ring buffer
End-to-end	<55 ms	SIC on critical path when invoked
Injection cls.	<200 ms	Async; not on critical path

Table 5: Per-component latency targets. End-to-end is <5 ms when the SIC is not invoked (majority of calls).

6 Open Research Problems

Cross-session taint via persistent memory. AgentArmor [18] correctly identifies that single-round taint analysis is insufficient: attackers can taint agent state across sessions via write-read chains, cache pollution, and persistent memory indexes. The CTT's per-session taint vector does not capture this. A complete taint model must track provenance through persistent memory (vector DB, filesystem, key-value stores) across sessions—particularly hard in multi-agent deployments where one agent's memory is another's tool result.

Policy verification for mixed symbolic-neural systems. The Capability Manifest is verifiable with standard formal methods; the SIC is a neural network. Can we verify that a (Manifest, SIC)

pair satisfies a stated safety property? Approaches from certified defenses, abstract interpretation, and randomized smoothing may be applicable, but compositionality between discrete rule-based policy and a probabilistic neural component remains unsolved. Even weaker guarantees—probability bounds on false-negative rates as a function of τ_{call}—would be practically valuable.

Multi-agent trust propagation. Current AgentBPF conservatively treats all inter-agent messages as EXTERNAL ($\tau = 0.7$). A principled trust propagation model would allow trust to flow through a delegation graph—analogous to SPIFFE trust bundles in service meshes—with revocation semantics if the delegatee processes untrusted data. Formalizing this is complicated by the non-determinism of LLM reasoning: the same delegation instruction may produce different behavior on different calls.

Autonomous Action Budget calibration. The budget B is AgentBPF's most operationally impactful parameter. Set too low: constant human interruptions eliminate productivity. Set too high: large blast radius for successful injections. The right calibration depends on task domain, reversibility distribution of tool calls, SIC quality, and operator risk tolerance. A principled, data-driven calibration approach—perhaps as Bayesian optimization over historical trajectory data—would substantially increase adoption of this primitive.

7 Related Work

Prompt injection detection. Greshake et al. [8] established indirect prompt injection as a fundamental attack class. Hung et al. [10] (Attention Tracker) demonstrated attention-based detection via the distraction effect (Section 2.3). PromptArmor [15] and SecAlign [2] represent the training-based direction—fine-tuning the backend LLM for injection robustness—complementary to AgentBPF but providing no runtime enforcement guarantees.

Runtime agent security. Ruan et al. [17] propose a dual-agent LLM safety evaluator for sandbox enforcement. AgentArmor [18] provides PDG-based formal taint for white-box settings. OS-level sandboxing (seccomp, namespaces, containers) operates at the syscall level and cannot evaluate semantic intent; AgentBPF operates at the tool-call level and cannot prevent OS-level escape. Production deployments should combine both layers.

Industrial agent governance. Microsoft's governance toolkit [12], Cisco AI Defense [3], and TrojAI Defend for MCP [19] represent industrial enforcement approaches. All validate the commercial demand for the enforcement-layer architecture. None publishes a generalizable framework with formal semantics, open research questions, or the taint-conditioned semantic layer that differentiates AgentBPF.

eBPF infrastructure. McCanne & Jacobson [11] introduced BPF; Gregg [9] provides the comprehensive reference. Wright et al. [20] established Linux Security Modules—the closest analog to SIC enforcement. Practitioners have discussed using actual eBPF for MCP server sandboxing at the syscall level; AgentBPF applies the design philosophy one full abstraction level higher, where semantic intent can be evaluated.

Capability-based security. The Capability Manifest is grounded in capability-based security theory [7, 13]. OPA (Open Policy Agent) is the closest existing policy language; the Capability Manifest is a domain-specific instantiation for the agent tool-call domain. The Autonomous Action Budget—a finite token of autonomous actions before a human checkpoint—does not appear in prior capability security literature to our knowledge.

8 Conclusion

We have presented AgentBPF, a reference architecture for runtime policy enforcement in agentic AI systems. The central claim is architectural: the most important security gap in current agentic systems is not a model alignment deficit but a *missing control plane*—the boundary between LLM reasoning and tool execution lacks principled policy, provenance tracking, and audit infrastructure.

AgentBPF addresses this gap with four composable primitives (TII, CTT, SIC, AARB), a declarative Capability Manifest, and an Autonomous Action Budget for bounding unchecked autonomy. The primary contribution is the architecture of integration: how these primitives connect taint provenance through structural policy to semantic verification to behavioral telemetry in a single enforcing pipeline.

The open problems in Section 6 define a concrete research agenda. Future work can implement and empirically evaluate these components; this paper specifies the architecture those implementations should instantiate. As agentic systems increasingly mediate real-world actions, the question is no longer whether runtime governance is needed, but what architectural form it should take.

References

[1] Y. Bai et al. Constitutional AI: Harmlessness from AI feedback. *Anthropic Technical Report*, 2022.

[2] J. Chen et al. SecAlign: Defending against prompt injection with preference optimization. *arXiv:2410.05451*, 2025.

[3] Cisco. Cisco redefines security for the agentic era with AI Defense expansion. *Cisco Newsroom*, February 2026.

[4] J. Clause, W. Li, and A. Orso. Dytan: A generic dynamic taint analysis framework. In *Proceedings of ISSTA*, 2007.

[5] CrowdStrike. What security teams need to know about OpenClaw. *CrowdStrike Blog*, 2026.

[6] E. Debenedetti et al. AgentDojo: A dynamic environment to evaluate attacks and defenses for LLM agents. *arXiv:2406.13352*, 2024.

[7] J. B. Dennis and E. C. Van Horn. Programming semantics for multiprogrammed computations. *Commun. ACM*, 9(3):143–155, 1966.

[8] K. Greshake et al. Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In *AISec Workshop at CCS*, 2023.

[9] B. Gregg. *BPF Performance Tools: Linux System and Application Observability*. Addison-Wesley, 2019.

[10] K.-H. Hung et al. Attention Tracker: Detecting prompt injection attacks in LLMs. *arXiv:2411.00348*, NAACL 2025 Findings, 2024.

[11] S. McCanne and V. Jacobson. The BSD packet filter: A new architecture for user-level packet capture. In *USENIX Winter Conference*, 1993.

[12] Microsoft. agent-governance-toolkit: AI agent governance toolkit. https://github.com/microsoft/agent-governance-toolkit, 2025.

[13] M. Miller, K.-P. Yee, and J. Shapiro. Capability myths demolished. *Technical Report SRL2003-02*, Systems Research Lab, 2003.

[14] J. Newsome and D. Song. Dynamic taint analysis for automatic detection, analysis, and signature generation of exploits on commodity software. In *Proceedings of NDSS*, 2005.

[15] PromptArmor. Simple yet effective prompt injection defenses. *arXiv:2507.15219*, 2025.

[16] CMU PDL. Defending LLM agents against prompt injection and privacy leakage (RTBAS). *CMU PDL Technical Report*, 2024.

[17] S. Ruan et al. Identifying the risks of LM agents with an LM-emulated sandbox. *arXiv:2309.15817*, 2024.

[18] AgentArmor Authors. Securing large language model agents via structured graph abstraction. *arXiv:2508.01249*, 2025.

[19] TrojAI. TrojAI Defend for MCP: Real-time security and policy enforcement for agentic AI workflows. *Help Net Security*, November 2025.

[20] C. Wright et al. Linux security modules: General security support for the Linux kernel. In *USENIX Security Symposium*, 2002.